

B. Tech. Project AY 2026-27

CA 1 Presentation

Project ID: INT2

Project: Porting ROCm to Consumer AMD GPUs



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Guides: Dr. Shruti Sunnad & Dr. Madhvi Saxena

Industry Mentors:

Rahul Poruri (CEO, FOSS United)

&

Bhooshan Agalgatti (Sr. Director, Nvidia)

Member: Soham Kulkarni (23070126129)

Department of Artificial Intelligence & Machine Learning

Project evolution and reporting window

- Current work targets g_{fx}90c, with workflow validation planned on g_{fx}902.
- Reporting period: 26-06-26 to 31-07-26 across two progress reports.
- Technical evidence is documented through eight project blog articles published on Substack.

AMD vis-à-vis GPUs #1

So.. what are they doing about Nvidia's moat?

"In a gold rush, those selling the shovels make the most money".

NETHERQUARK.SUBSTACK.COM

AMD vis-à-vis GPUs #2

Installing ROCm & PyTorch on Fedora 43 natively

FP32 compute (h AMD vis-a-vis GPUs #3

Is GPU spoofing a solution?

Dc (native)

AMD vis-à-vis GPUs #4

Running first party AMD benchmarks

Response	Percentage
Yes, the current system is the best way to run the country	55%
No, the current system is not the best way to run the country	45%

NETHERQUARKS310

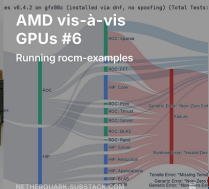
AMD vis-à-vis GPUs #5

Introduction to GPU Programming



AMD vis-à-vis GPUs #6

Running rocm-examples



NETHERQUARK.SUBSTACK.COM

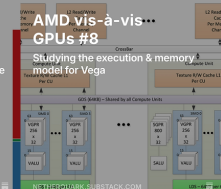
AMD vis-à-vis GPUs #7

Studying the compiler stack by building rocm-examples from source

NETHERQUARK.SUBSTACK.COM

AMD vis-à-vis GPUs #8

Studying the execution & memory model for Vega



Introduction

- Conducted as an employee of *FOSS United Foundation*.
- ROCm is AMD's CUDA
- Primary target: gfx90c (Ryzen 5 5650U); validation target: gfx902 (Ryzen 5 2200G).
- Goal: develop a reproducible workflow for adding ROCm support.



Problem statement

- Native HIP execution succeeds, but packaged ROCm libraries lack complete gfx90c support.
- Missing rocBLAS/Tensile artifacts prevent higher-level workloads such as PyTorch.
- Community spoofing enables evaluation but is not a production-grade solution.
- A reproducible recovery workflow for unsupported architectures is currently unavailable.

Objectives of the project

- Recover functional ROCm support for gfx90c using reproducible methods.
- Benchmark every recovery stage against the native software stack & against spoofing.
- Validate the methodology on gfx902.
- Generalise the workflow for other unsupported AMD architectures.

Project Timeline

- Jun 2026: Verify feasibility of project
- Jul 2026: Port 1 HIP application from gfx900 to gfx90c
- Aug 2026: Develop expertise in GCN Assembly, HIP, ROCm build system
- Sep 2026: Finish porting rocm-examples
- Oct 2026: Finish porting full ROCm stack
- Nov 2026: Finish porting PyTorch / Dockerised PyTorch
- Dec 2026: Establish a reproducible recovery workflow for gfx90c.
- Jan 2027: Benchmark port vs spoofing, run profiling & optimise fixes. Check & resolve regressions as per ROCm test suite.
- Feb 2027: Validate approach on gfx902
- Mar 2027: Set up CI/CID for all unsupported AMD GCN GPUs.
- Apr-Jul 2027: Contribute upstream to Mojo, LLVM, ROCm, PyTorch, and vLLM with RDNA 4 development platform.

Literature review – ROCm software stack

- ROCm Documentation (rocm.docs.amd.com/...)
- HIP Programming Model
(rocm.docs.amd.com/.../programming_model)
- HIP Programming Patterns
(rocm.docs.amd.com/.../programming-patterns)
- HIP SAXPY Tutorial (rocm.docs.amd.com/.../saxpy)
- HIPIFY (github.com/ROCm/HIPIFY)
- ROCm Examples (github.com/ROCm/rocm-examples)
- ROCm Libraries (github.com/ROCm/rocm-libraries)
- rocBLAS (github.com/ROCm/rocBLAS)
- hipBLAS (github.com/ROCm/hipBLAS)
- ROCm Platform (github.com/ROCm/ROCm)
- ROCm LLVM Fork (github.com/ROCm/llvm-project)

Literature review – Compiler, build system and upstream development

- ROCm Libraries PR #8139 (github.com/R0Cm/.../pull/8139)
- TheRock Issue #3818 (github.com/R0Cm/.../issues/3818)
- TheRock Build System (github.com/R0Cm/TheRock)
- ROCm GitHub Organisation (github.com/R0Cm)
- RadeonOpenCompute (github.com/RadeonOpenCompute)
- LLVM AMDGPU Backend (github.com/llvm/.../AMDGPU)
- LLVM Clang (github.com/llvm/.../clang)
- ROCm LLVM Toolchain (github.com/R0Cm/llvm-project)
- AMD GCN ISA Reference (gpuopen.com/.../gcn-assembly)
- GPUOpen (gpuopen.com)

Literature review – GPU architecture and supporting references

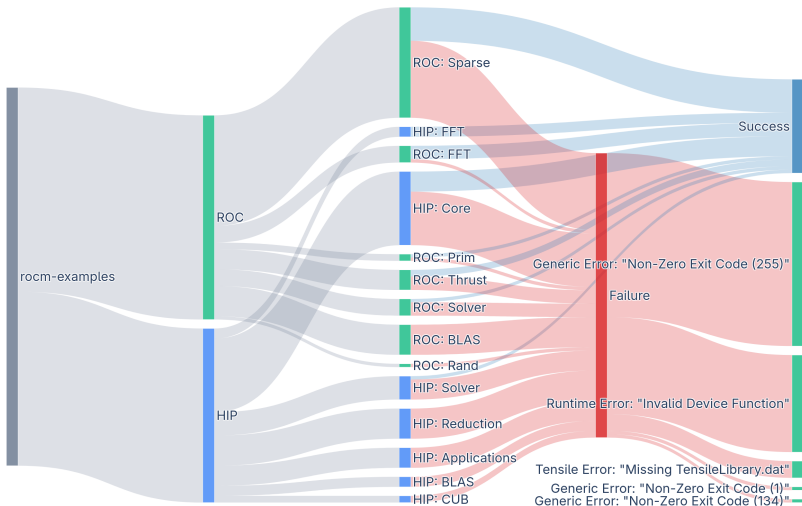
- Graphics Processing Unit
(wikipedia.org/.../Graphics_processing_unit)
- Geometry Engine (wikipedia.org/.../Geometry_Engine)
- Transformation Matrix
(wikipedia.org/.../Transformation_matrix)
- GPUOpen Technical Articles (gpuopen.com/learn)
- Project Research Notes (docs.google.com/...)
- Azu (github.com/Netherquark/azu)
- PuneFOSS CFP (fossunited.org/.../2lhj0bntp3)
- ROCm Examples Repository (github.com/ROCm/rocm-examples)
- RadeonOpenCompute Organisation
(github.com/RadeonOpenCompute)
- HIPIFY Repository (github.com/ROCm/HIPIFY)

Gap in the research, technology, and methodology

- No documented end-to-end workflow for recovering unsupported ROCm targets.
- Packaged ROCm components omitted gfx90c-specific code generation and runtime data.
- Recovery knowledge was fragmented across documentation, issue trackers, pull requests, and community discussions.
- No methodology existed to validate recovery, benchmark correctness, and reproduce results across GPU generations.

Gap in the research, technology, and methodology

rocm-examples v6.4.2 on gfx90c (installed via dnf, no spoofing) (Total Tests: 113)



Description of the proposed solution

- Establish a staged recovery workflow from native binaries to complete source builds.
- Validate every layer using HIP applications, rocm-examples, and PyTorch workloads.
- Isolate compiler, runtime, library, packaging, and hardware-specific failures independently.
- Produce a reproducible workflow transferable to other unsupported AMD GCN GPUs.

Requirement analysis

```
=== Operating System ===
Fedora Linux 43 (Workstation Edition)

=== Kernel ===
6.19.13-200.fc43.x86_64

=== CPU ===
Model name: AMD Ryzen 5 PRO 5650U with Radeon Graphics

=== GPU ===
07:00.0 VGA compatible controller: Advanced Micro Devices, Inc. [AMD/ATI] Cezanne [Radeon Vega Series / Radeon Vega Mobile Series] (rev d2)

=== ROCm Target ===
Name: gfx90c
Uuid: GPU-XX
Marketing Name: AMD Radeon Graphics
Vendor Name: AMD
Feature: KERNEL_DISPATCH
--
Name: amdgcN-amd-amdhsa--gfx90c:xnack+
Machine Models: HSA_MACHINE_MODEL_LARGE
Profiles: HSA_PROFILE_BASE
Default Rounding Mode: NEAR
Default Rounding Mode: NEAR
```

Technology Stack



Native ROCm baseline

- Installed ROCm 6.4.4 on Fedora 43 for gfx90c.
- Verified HIP compilation and execution using native ROCm binaries.
- Identified PyTorch failures caused by missing gfx90c rocBLAS Tensile data.
- Evaluated HSA architecture spoofing as an initial compatibility baseline.
- Ran Fedora provided rocm-examples as a baseline.

Source recovery

- Built rocm-examples from source using the ROCm 6.4.4 toolchain.
- Investigated missing main_gfx90c.s assembly generation.
- Extended the assembly_to_executable workflow for gfx90c.
- Traced compilation from HIP source through LLVM IR to AMDGPU assembly.

Failure analysis

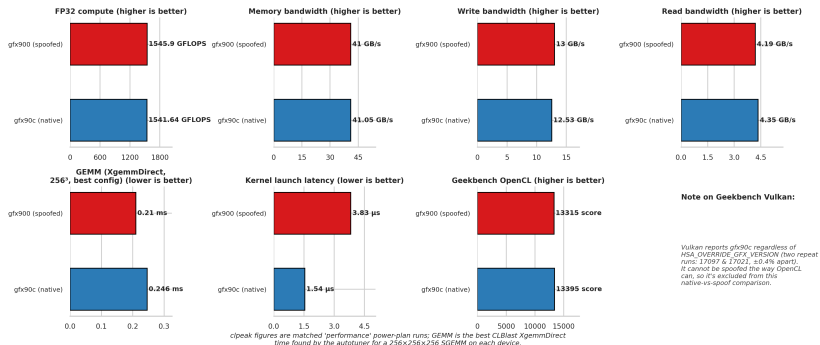
- Classified failures into packaging, metadata, missing Tensile data, and hardware limitations.
- Compared behaviour across native, spoofed, and source-built environments.
- Validated each recovery stage using HIP applications and rocm-examples.
- Recorded reproducible failure cases for subsequent recovery work.

Benchmark evaluation

- Evaluated memory bandwidth using CLBlast and clpeak.
- Measured matrix multiplication performance using rocBLAS benchmark clients.
- Compared application behaviour under native and spoofed execution.
- Used benchmark results to validate recovered functionality and identify remaining regressions.

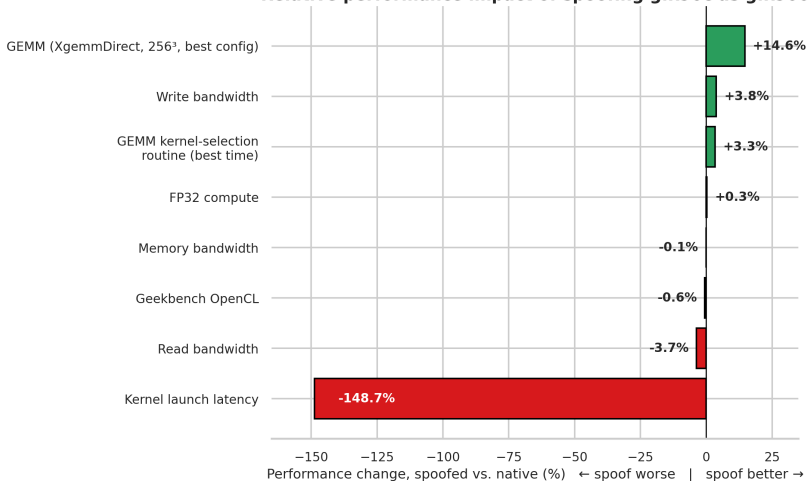
Benchmark evaluation

gfx90c native vs. gfx900-spoofed: overall hardware & GEMM performance



Benchmark evaluation

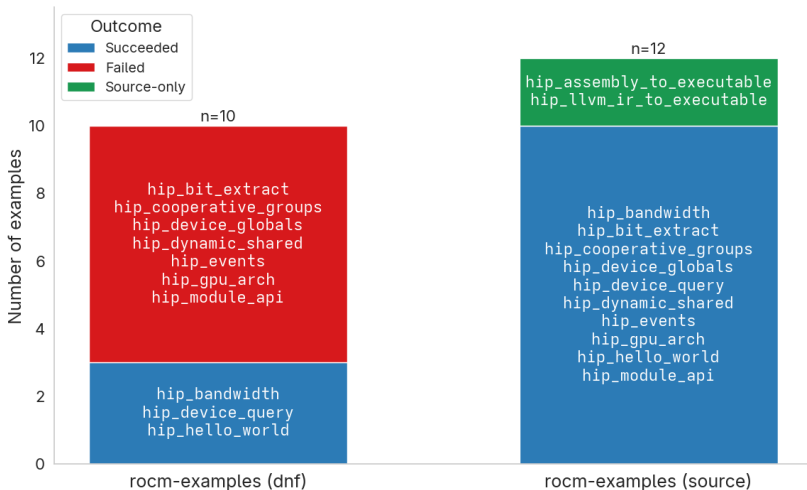
Relative performance impact of spoofing gfx90c as gfx900



Compute throughput and bandwidth are within noise ($\pm$ a few %); the two clear, consistent effects of spoofing are much higher kernel-launch latency and, interestingly, a modestly faster tuned GEMM kernel.

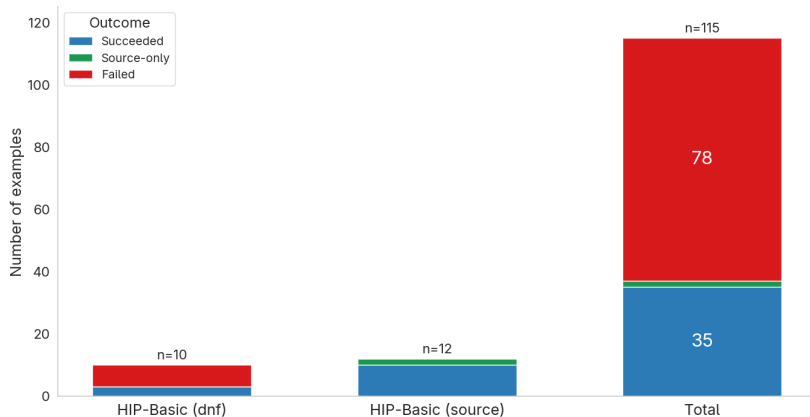
Benchmark evaluation

DNF vs Source Build (HIP-Basic subset)



Benchmark evaluation

Overall Progress



Intended project outcome

- A reproducible workflow for recovering unsupported AMD GCN GPUs under ROCm.
- Complete gfx90c enablement across the ROCm software stack.
- Validation of the recovery methodology on gfx902.
- Benchmark-backed evaluation of native, spoofed, and recovered software stacks.
- Documentation suitable for upstream contribution and long-term maintenance.

Conclusion

- Native ROCm functionality on unsupported hardware was successfully characterised.
- Compiler, runtime, packaging, and library limitations were systematically isolated.
- A staged recovery methodology was established for unsupported ROCm targets.
- The proposed workflow provides a foundation for future upstream enablement of legacy AMD GPUs.

References

- AMD ROCm Documentation
<https://rocm.docs.amd.com/>
- AMD ROCm Programming Guide
<https://rocm.docs.amd.com/projects/HIP/en/latest/>
- HIP Programming Model
https://rocm.docs.amd.com/projects/HIP/en/latest/understand/programming_model.html
- HIP Programming Patterns
<https://rocm.docs.amd.com/projects/HIP/en/latest/tutorial/programming-patterns.html>
- HIP SAXPY Tutorial
<https://rocm.docs.amd.com/projects/HIP/en/latest/tutorial/saxpy.html>
- ROCm GitHub Organisation
<https://github.com/ROCm>
- ROCm Platform Repository
<https://github.com/ROCm/ROCm>
- ROCm Libraries Repository
<https://github.com/ROCm/rocm-libraries>
- ROCm Libraries Pull Request #8139
<https://github.com/ROCm/rocm-libraries/pull/8139>
- ROCm Examples Repository
<https://github.com/ROCm/rocm-examples>

References

- rocBLAS Repository
<https://github.com/ROCm/rocBLAS>
- hipBLAS Repository
<https://github.com/ROCm/hipBLAS>
- TheRock Repository
<https://github.com/ROCm/TheRock>
- TheRock Issue #3818
<https://github.com/ROCm/TheRock/issues/3818>
- ROCm LLVM Fork
<https://github.com/ROCm/llvm-project>
- LLVM Project
<https://github.com/llvm/llvm-project>
- LLVM AMDGPU Backend
<https://github.com/llvm/llvm-project/tree/main/llvm/lib/Target/AMDGPU>
- GPUOpen
<https://gpuopen.com/>
- AMD GCN Assembly Article
<https://gpuopen.com/learn/amd-gcn-assembly-cross-lane-operations/>
- RadeonOpenCompute
<https://github.com/RadeonOpenCompute>

References

- HIPIFY Repository
<https://github.com/ROCm/HIPIFY>
- Graphics Processing Unit
https://en.wikipedia.org/wiki/Graphics_processing_unit
- Geometry Engine
https://en.wikipedia.org/wiki/Geometry_Engine
- Transformation Matrix
https://en.wikipedia.org/wiki/Transformation_matrix
- Azu
<https://github.com/Netherquark/azu>
- PuneFOSS 2025 CFP
<https://fossunited.org/c/pune/punefoss4/cfp/2lhj0bntp3>
- AMD vis-à-vis GPUs Blog Series (#1–#8)
- Progress Report No. 1
- Progress Report No. 2

31 July 2026 CA-1 Presentation

